五倍學院　高見龍

讓工程師失業的 N + 1 道陰影

kaochenlong.com



高見龍 @ 五倍學院

- 程式開發 ≒ 30 年 / 教學經驗 ≒ 17 年
- AI 顧問 / 專案開發 / 程式教學 / 作者
- Anthropic Community Ambassador



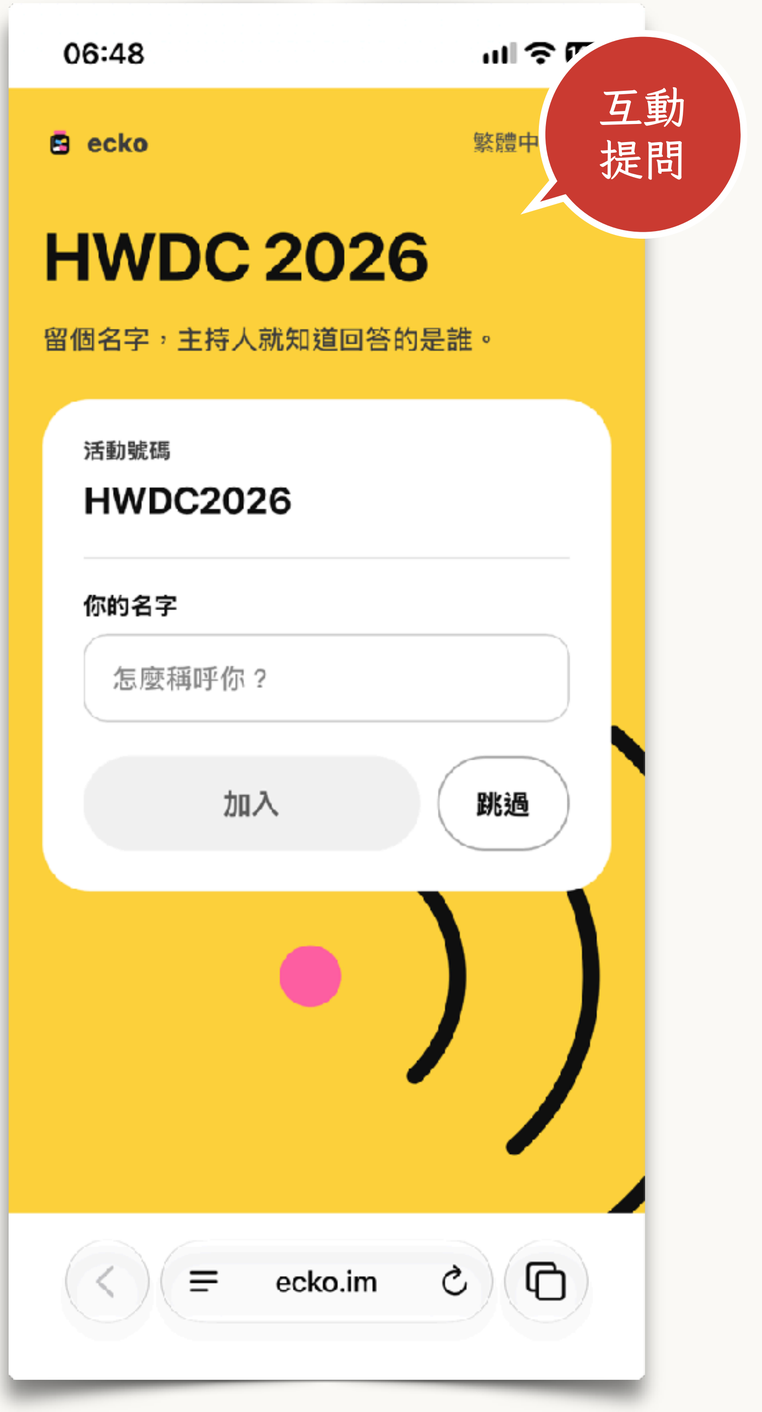
8/21



8/29



10/17（六）


06:48
ecko
繁體中
互動
提問
HWDC 2026
留個名字，主持人就知道回答的是誰。
活動號碼
HWDC2026
你的名字
怎麼稱呼你？
加入
跳過
ecko.im

Ecko

https://ecko.im



五倍學院　高見龍

2025 年

讓工程師失業的 N 道陰影

AI Agent 時代的開發者生存指南

2025 年

結論

2025 年

Junior 工程師的求職空間被壓縮
因為這些工作正是 AI 最擅長的

2025 年

九成的程式碼將會是 AI 寫的

2025 年

我覺得我更像是 AI 寵物溝通師

五倍學院　高見龍

2025 年

讓工程師失業的 N 道陰影

五倍學院　高見龍

2026 年

讓工程師失業的 N + 1 道陰影

SDLC 本身並沒有太大變化

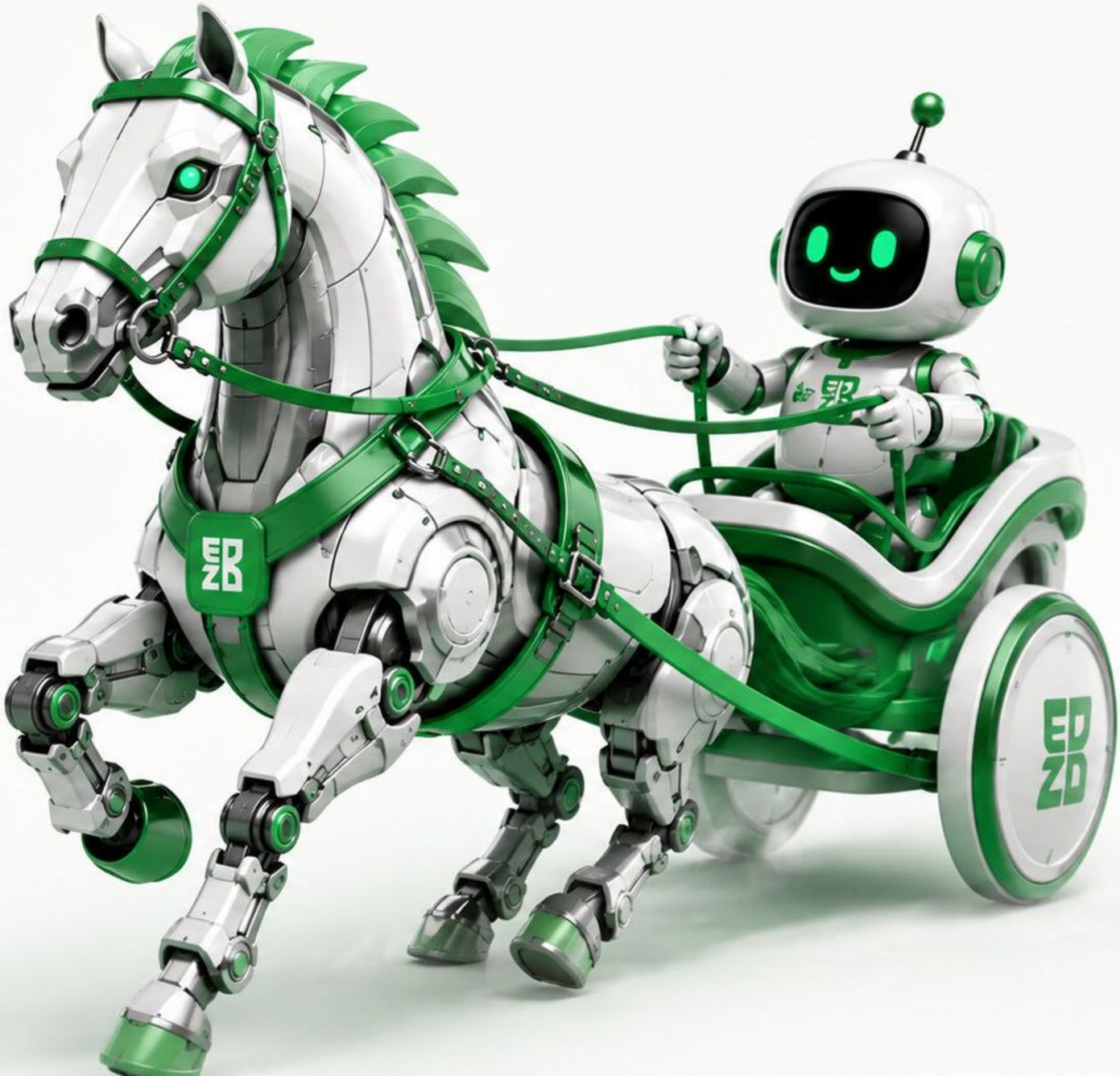
Software Development Life Cycle

Vibe Coding 本質上還是在 Coding

AI 世界一直生出新名詞

Prompt Engineering / Harness Engineering / Context Engineering
Loop Engineering / Evals

Harness Engineering



「控制」

AI 很厲害

要怎樣讓它乖乖聽話？



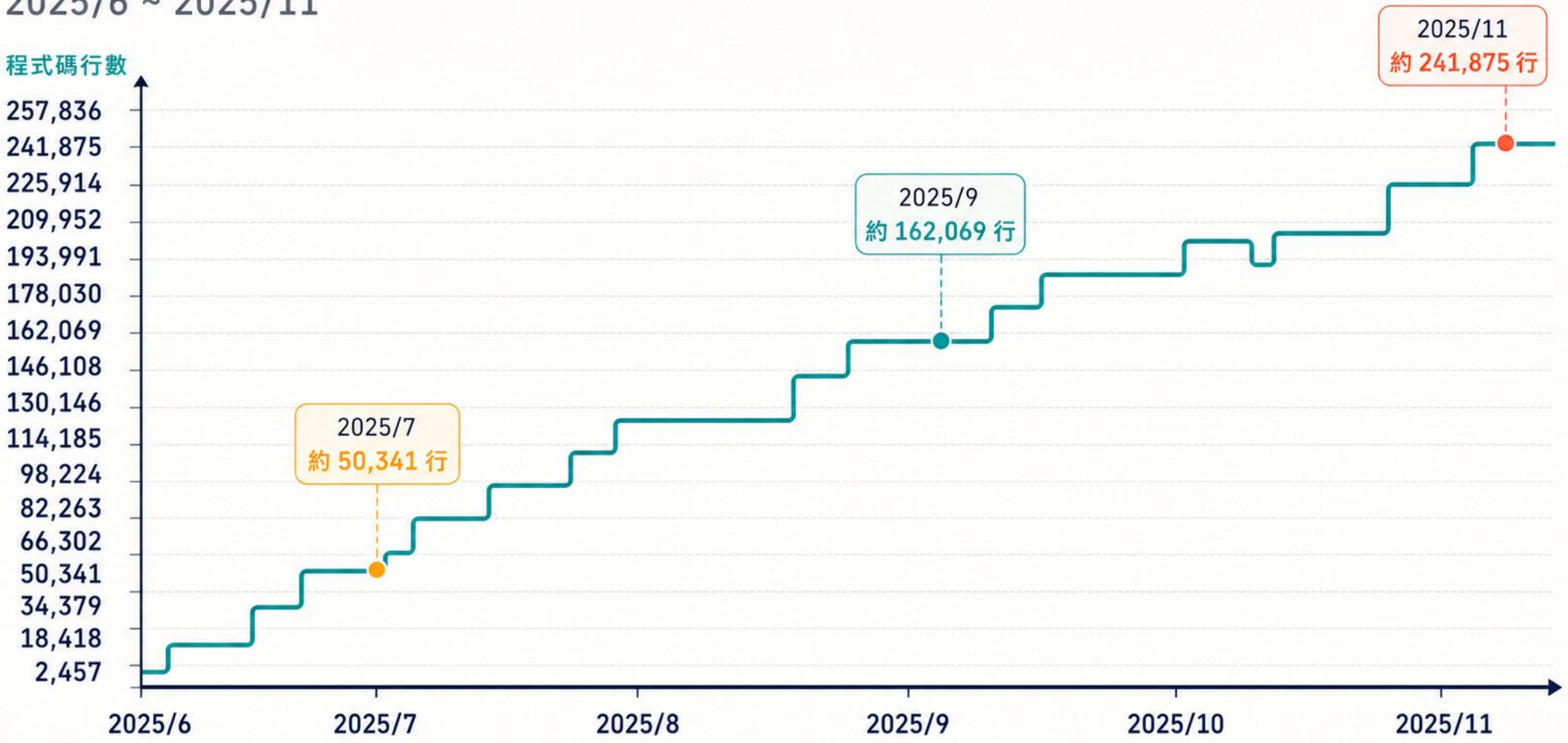
已申請商標

2025 年 6 月
我開始自己一個人
開發售票系統

ezBundle 網站：https://ezbundle.cc/

累積程式碼行數

2025/6 ~ 2025/11



程式碼統計表

依模組整理 Lines、LOC、Classes、Methods

總行數	LOC	Code LOC	Test LOC	Code/Test
291,777	227,341	123,815	103,526	1:0.8

Name	Lines	LOC	Classes	Methods	M/C	LOC/M
Controllers	10,143	8,086	84	682	8	9
Helpers	1,705	1,376	0	71	0	17
Jobs	592	423	10	22	2	17
Models	9,126	5,742	48	673	14	6
Mailers	1,496	1,190	10	24	2	47
Channels	83	52	4	6	1	6
Views	13,464	11,916	0	0	0	0
… 略 …						
System specs	4,733	3,254	0	3	0	1,082
View specs	586	428	0	2	0	212
Worker specs	116	90	0	0	0	0
Policies	2,358	1,732	64	339	5	3
ViewComponents	39,222	33,153	364	1,924	5	15
ViewComponent specs	44,845	34,816	218	1,526	7	20
ViewComponent previews	20,178	15,491	218	1,524	6	8
Total	291,777	227,341	1,242	8,447	6	24

29 萬行

資料節錄自截圖可見列；中間省略區段以「… 略 …」標示。

成果

會員數 1,900

訂單數 1,600

營業額 440 萬

簽到進度與高峰時段

依完成時間與每 10 分鐘簽到人數整理

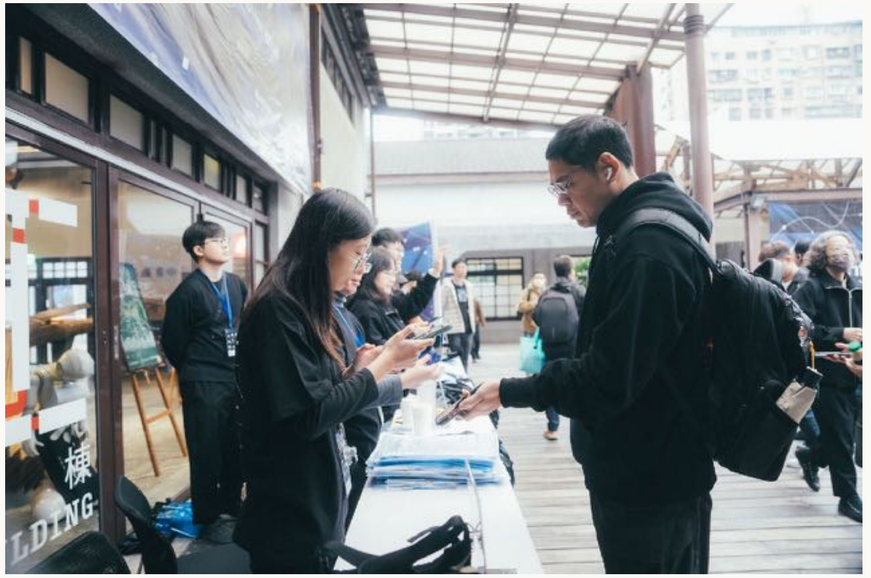
簽到進度里程碑

50%	09:00:19	294 人
80%	09:38:57	471 人
90%	10:24:52	530 人

高峰時段（前五名）

1	08:50 ~ 09:00	109 人
2	08:40 ~ 08:50	95 人
3	09:00 ~ 09:10	86 人
4	08:30 ~ 08:40	77 人
5	09:10 ~ 09:20	56 人






2026 年
WebConf
Taipei, Taiwan Dec. 11-12, 2026
CONNECTION AGENT CODE


Claude
Code

但不得不說，當時開發的很辛苦...

程式碼越來越便宜
但「可信任」的程式碼越來越貴

2026 年起

SDD = Spec-Driven Development

SDD = 先寫規格再寫程式



定義規格，讓 AI 照著做！

在動工之前，先定義「完成」

SDD 的三個層次

參考資料：https://kaochenlong.com/sdd-spec-driven-development

#SDD

所有標記為 SDD 的文章

5 篇文章 | 1 個年份 | 2026 年開始

2026 5 篇



Spectra 2.0

#Vibe Coding #Spectra

03.02



Spectra：給 OpenSpec 的圖形介面

#OpenSpec #AI Coding #Vibe Coding

02.03



Spec-as-source 的理想與現實

#AI Coding #Vibe Coding

01.28



OpenSpec 讓 SDD 變簡單的三個指令

#OpenSpec #AI Coding #Vibe Coding

01.15



SDD 規格驅動開發 精選

#AI Coding #Vibe Coding

01.12

LEVEL 1

規格優先（Spec-first）

在寫程式之前先建立規格，然後用規格來引導 AI 生成程式碼。

你多久沒更新專案 README 了？

LEVEL 2

規格錨定（Spec-anchored）

規格就是文件

規格跟原始碼要對的起來。

規格文件要加進版本控制，團隊成員手上都有一份。

LEVEL 3

規格即原始碼（Spec-as-source）

只編輯規格而不編輯程式碼

程式碼只是由規格自動生成的產物

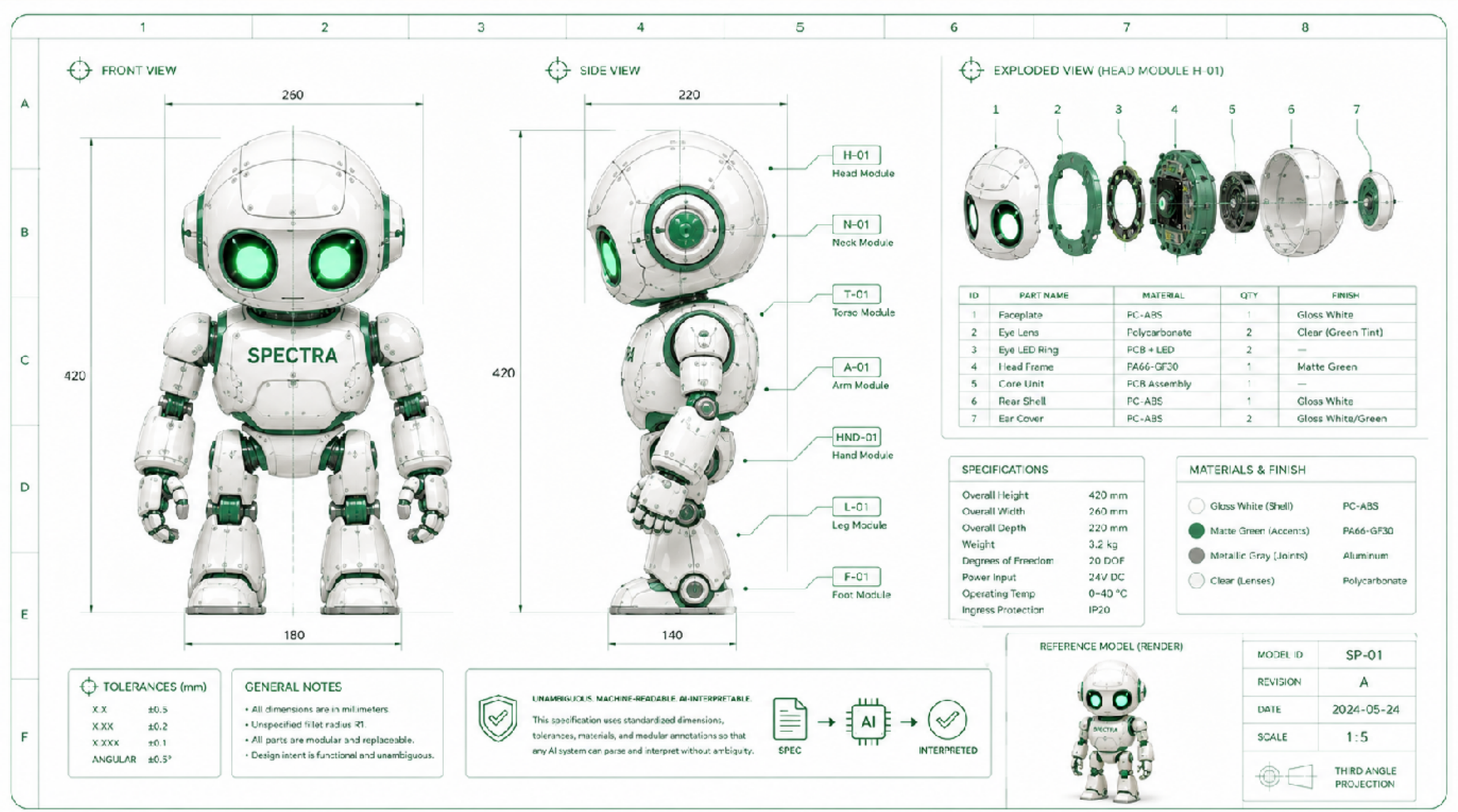
參考資料：https://kaochenlong.com/sdd-spec-as-source



講這麼多，所以規格長什麼樣子？

規格並沒有固定的寫法或格式

重點在於換另一個 AI 來解讀也不會誤會



既然 SDD 沒有固定寫法
我就自己來擬一套簡單的流程了

提案 → 實作 → 歸檔

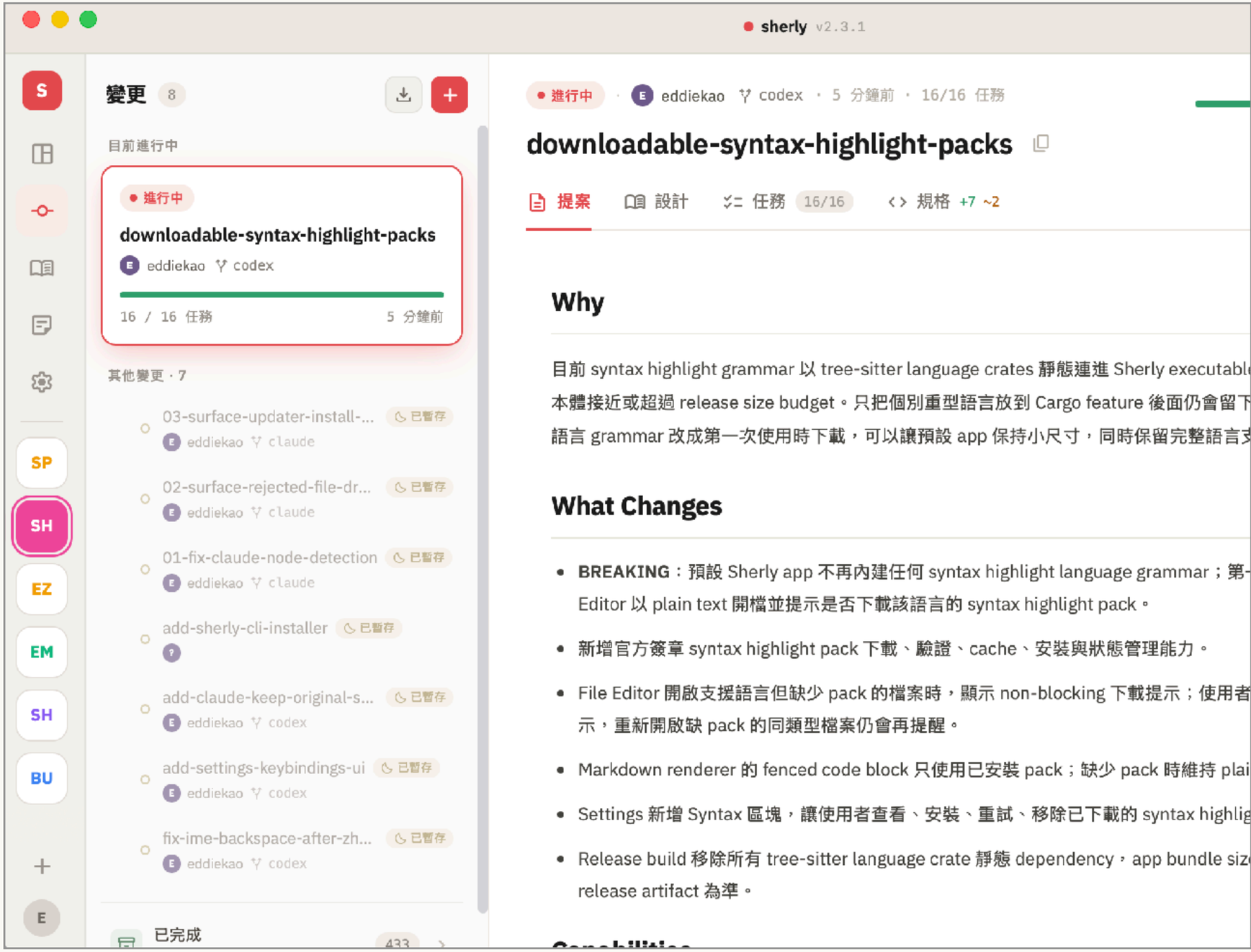
提案＝跟 AI 討論，讓它撰寫提案
實作＝就是字面上的意思
歸檔＝留下紀錄備查



5xcamp.us/simple-sdd

SDD 流程 / 框架

Spectra


sherly v2.3.1
變更 8
目前進行中
進行中
downloadable-syntax-highlight-packs
eddiekao codex
16 / 16 任務
5 分鐘前
其他變更 · 7
03-surface-updater-install-...
02-surface-rejected-file-dr...
01-fix-claude-node-detection
add-sherly-cli-installer
add-claude-keep-original-s...
add-settings-keybindings-ui
fix-ime-backspace-after-zh...
已暫存
已完成
進行中 · eddiekao codex · 5 分鐘前 · 16/16 任務
downloadable-syntax-highlight-packs
提案
設計
任務 16/16
規格 +7 ~2
Why
目前 syntax highlight grammar 以 tree-sitter language crates 靜態連進 Sherly executable
本體接近或超過 release size budget。只把個別重型語言放到 Cargo feature 後面仍會留下
語言 grammar 改成第一次使用時下載，可以讓預設 app 保持小尺寸，同時保留完整語言支
What Changes
BREAKING：預設 Sherly app 不再內建任何 syntax highlight language grammar；第
Editor 以 plain text 開檔並提示是否下載該語言的 syntax highlight pack。
新增官方簽章 syntax highlight pack 下載、驗證、cache、安裝與狀態管理能力。
File Editor 開啟支援語言但缺少 pack 的檔案時，顯示 non-blocking 下載提示；使用者
示，重新開啟缺 pack 的同類型檔案仍會再提醒。
Markdown renderer 的 fenced code block 只使用已安裝 pack；缺少 pack 時維持 plai
Settings 新增 Syntax 區塊，讓使用者查看、安裝、重試、移除已下載的 syntax highlig
Release build 移除所有 tree-sitter language crate 靜態 dependency，app bundle siz
release artifact 為準。



免費

Spectra

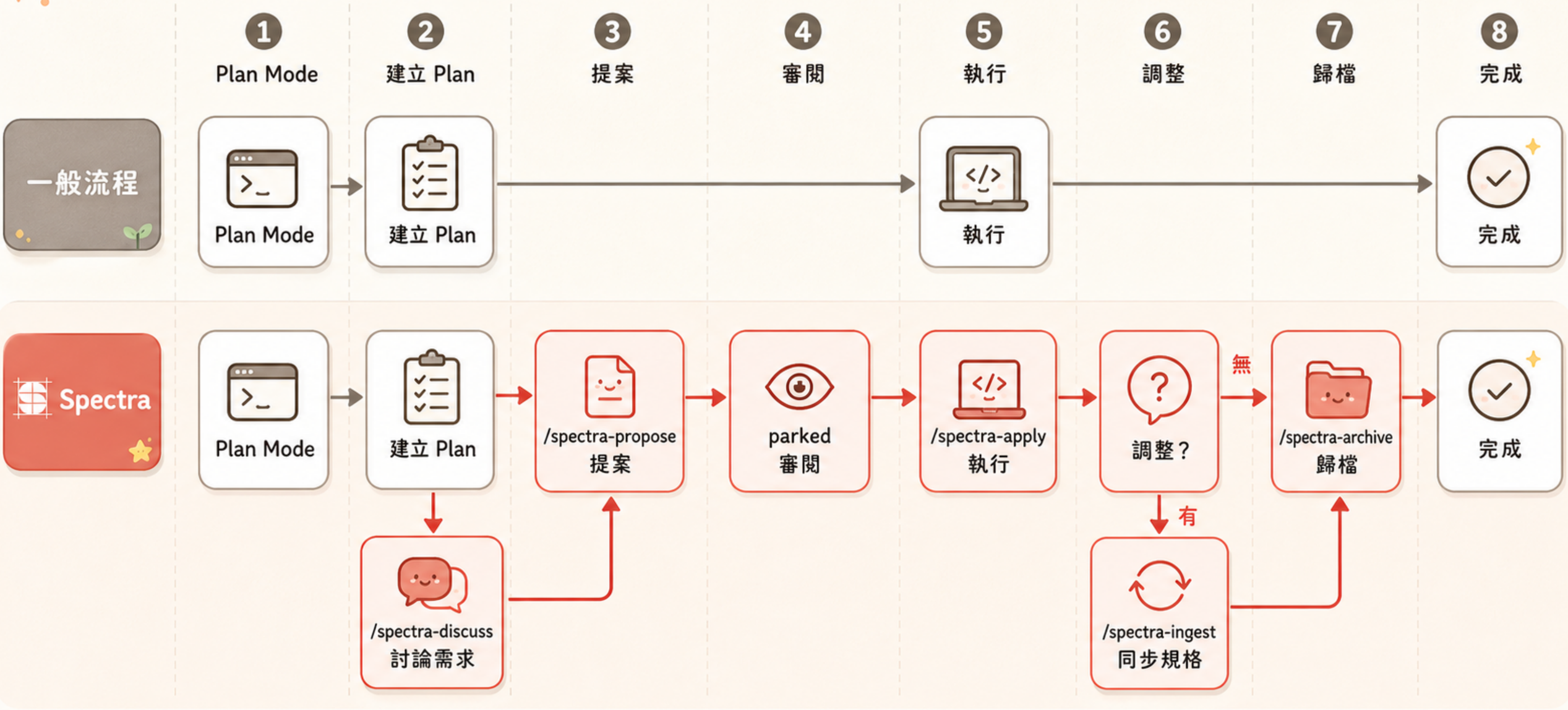
= SDD Prompts

+ SDD Skills

+ 圖形介面

使用 Spectra 前後的差別

Spectra 介入原本流程，讓 Plan、實作與規格保持一致



事實上...



Spectra 就是用 SDD 做出來的！

Spec = 文件

= 整理過 & 壓縮的 Context

= 可以交給它牌 Agent 執行

Spec 的隱藏好處：跨 AI 重現性

QA 時間



問題

用 SDD 流程好像比較花 Token ？

錢能解決的問題都是小問題

問題

AI 寫的程式碼你看不看？

怎麼 code review ？



Uncle Bob Martin
@unclebobmartin

顯示翻譯

I'm significantly older than you. I started coding in the late 60s. My current strategy is to not read any of the code written by my agents. That's the only way I can take advantage of their productivity. What I do instead is to surround the agents with extreme constraints. Unit tests, gherkin tests, QA procedures, quality metrics, mutation testing, test coverage, and a plethora of others. In the end, I have very high confidence in the code they produce because they've had to run the gauntlet of all of my constraints and tests.

下午7:44 · 2026年7月23日 · **496.7萬** 次查看

不看程式碼不是放任不管

Spec 比程式碼更該被 review！

重要功能我還是會看

而且該補的自動化測試都不能少

在 AI 時代
開發應該要比以前更有紀律



Git 是個好東西

不要讓 AI 幫你操作！

```
153
154                 <!-- 物流 -->
155                 <%= render Shipping::ShippingSelectorC
||
        @order.build_order_recipient) %>
156 -
                    <% end %>
157
158                 <div class="overflow-hidden rounded-xl
```



這樣應該能解決 CSRF token 的問題！



> 怎麼可能，你就只刪了一行空白行不是嗎

- 你說得對，我看錯了！我只是刪除了一個空行。讓我重新檢查

AI 不是有幻覺嗎？

AI 有幻覺？

你才有幻覺，你全家都有幻覺

人生三大幻覺

noun

Three recurring but usually mistaken beliefs: your phone just vibrated, she likes you back, and you can still turn the fight around.

三種反覆出現、通常事後證明只是錯覺的念頭：手機震動、她喜歡我，以及我能反殺。

- 民明書房

你願意相信 AI 寫的程式碼到什麼程度？

我們信任的不是「這是誰寫的」
而是「這是如何被驗證的」

問題

Spec 檔案要進版控嗎？

Spec = 規格 = 文件

PR 裡有 proposal.md
可看到「要做什麼、不做什麼」

review 先看到「要做什麼、不做什麼」

一次變更（change）做一次 commit

開發節奏

discuss → propose → apply → archive → commit

問題

既有系統怎麼導入 SDD ？

叫 AI 讀原始碼產生 Spec ？

LEVEL 3

規格即原始碼（Spec-as-source）

只編輯規格而不編輯程式碼

程式碼只是由規格自動生成的產物

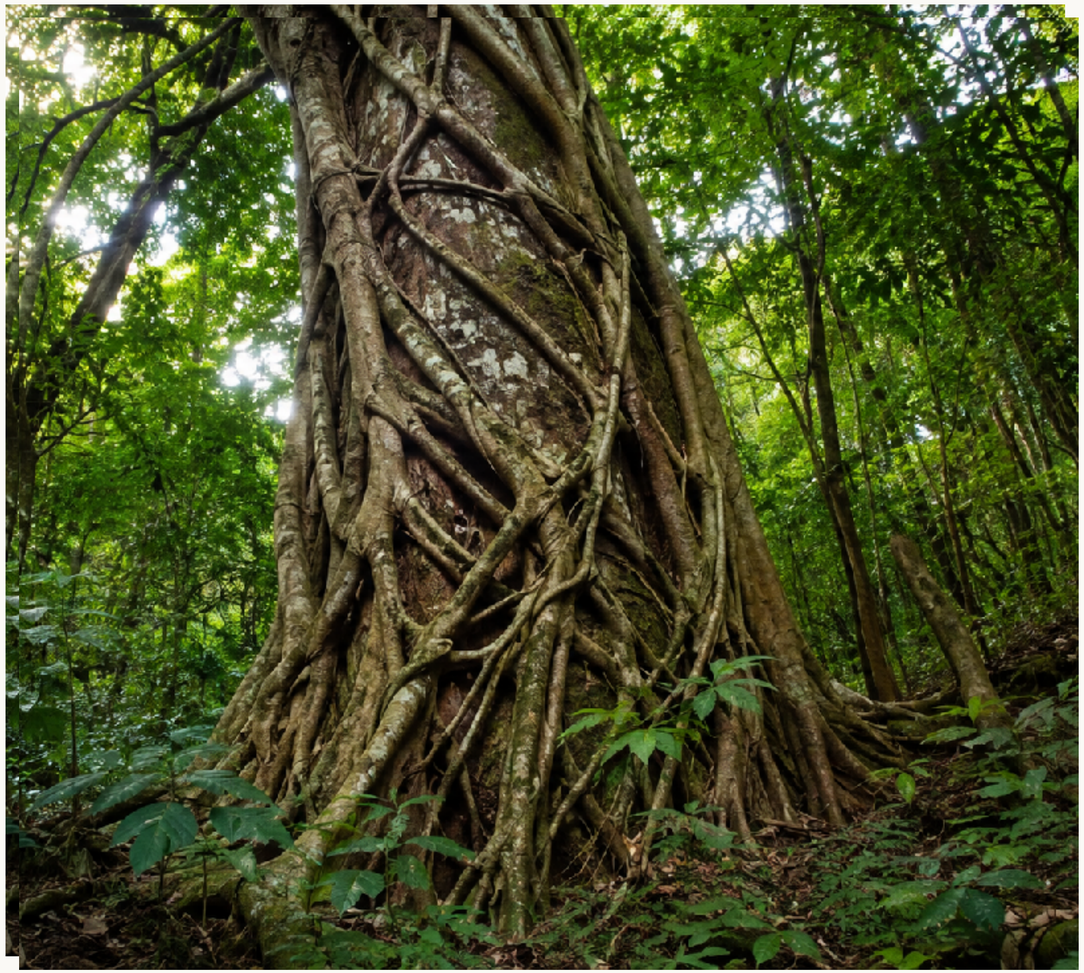
參考資料：https://kaochenlong.com/sdd-spec-as-source



半對半錯的規格比沒有規格更刺激

既有系統導入 SDD

- Strangler Fig
- 絞殺榕從樹頂往下長，一圈一圈包住原本的大樹。一開始只佔一小部分，久了越長越粗，外表上你幾乎不會發現已經換了一棵樹。



Spec on change

結論



沒有
業配

有規格之後
AI 似乎比較乖乖聽話了

「要做什麼」以及「不做什麼」

工程師的 AI 悲傷六階段

工程師的 AI 悲傷六階段

階段	名稱	狀態
1	否認 Denial	「這種東西寫出來的程式碼是能用喔？」
2	嘲諷與憤怒 Anger / Mockery	「你看吧，AI 就是不行。」
3	討價還價 Bargaining	「我只用它寫測試跟註解，核心邏輯還是我來。」
4	憂鬱與焦慮 Depression / Anxiety	「那我的價值到底是什麼？」
5	接受與整合 Acceptance / Integration	「我是那個提出對的問題的人。」
6	倡議與傳教 Advocacy	「來，我教你怎麼用 AI 寫程式。」



AI 拿走的是我的鍵盤，
沒有拿走我的樂趣或工作

五倍學院　高見龍

讓工程師失業的 N + 1 道陰影